

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

PATIALA, PUNJAB

DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING

CodeShield

ML-Based Code Vulnerability Detection and Risk
Assessment Platform

Evaluation Report

Submitted to: **Ms. Paramveer Kaur**

Course: UCS503

Degree: B.E. Computer Engineering

Team Members

Rhea Balamurugan	1024030152
Ishi Govil	1024030161
Shreya Kush	1024030718

Contents

1	Introduction	1
1.1	Background and Context	1
1.2	Problem Statement	1
1.3	Project Objectives	1
1.4	Scope	2
2	Software Development Life Cycle and Project Process	3
2.1	Selected SDLC Model	3
2.2	SDLC Diagram	3
2.3	Development Increments	3
3	Requirements Analysis	4
3.1	Functional Requirements	4
3.2	Non-Functional Requirements	4
4	System Architecture and Design	5
4.1	Use Case Diagram	5
4.2	High-Level System Architecture	5
4.3	Core Scanning Workflow	6
4.4	UML Class Diagram	7
4.5	Activity Diagram	7
5	Methodology and Design	8
5.1	Code Preprocessing	8
5.2	Static Analysis	8
5.3	Machine Learning Pipeline	8
5.3.1	TF-IDF Feature Extraction	8
5.3.2	Linear SVM Classifier	9
5.3.3	Probability Calibration and Confidence	9
5.4	Vulnerability Information and Risk Assessment	9
5.5	Result Fusion	9
6	Implementation	10
6.1	Frontend	10

6.2	Backend	10
6.3	Database	10
6.4	ML Model Integration	10
7	Testing and Evaluation	11
7.1	Testing Methodology	11
7.2	Performance Metrics	11
7.3	Results Table	11
7.4	Model Configuration	12
7.5	Confusion Matrix	12
8	Results and Analysis	13
8.1	Example Vulnerability Detection	13
8.2	Challenges and Mitigation	13
9	Conclusion and Future Work	14
9.1	Summary of Work	14
9.2	Key Findings	14
9.3	Future Work	14
9.4	Final Remarks	14

Chapter 1

Introduction

1.1 Background and Context

Modern software systems contain large volumes of source code and may contain security weaknesses that are difficult to identify through manual inspection alone. Vulnerabilities can remain hidden during normal execution and may only become visible after exploitation.

CodeShield is a software-security platform that combines static code analysis with machine learning. The system accepts source code, analyzes it, identifies selected vulnerability categories, estimates risk, and presents the findings through a developer-facing dashboard. The project aims to identify selected vulnerabilities rather than claim exhaustive detection of every possible weakness.

1.2 Problem Statement

CodeShield addresses the following problems:

- Large codebases are difficult to inspect manually.
- Security vulnerabilities may not produce visible runtime errors, and fixing them after deployment is costly.
- Different vulnerability classes require different detection strategies.
- Developers need prioritization so that critical findings can be addressed first.

1.3 Project Objectives

1. Develop a web-based interface for submitting source code.
2. Build a FastAPI backend that manages scan requests and analysis.
3. Combine static-analysis rules and machine-learning predictions to detect vulnerabilities.
4. Provide vulnerability type, severity, confidence, location, and explanation.
5. Store results and present them through a dashboard.

6. Evaluate the system using both ML and software-system metrics.

1.4 Scope

The project currently focuses on the following vulnerability categories:

- SQL Injection
- Command Injection
- Hardcoded Credentials
- Cross-Site Scripting (XSS)
- Insecure File Handling

The ML model works on Python code. Additional languages and vulnerability classes can be incorporated as future extensions.

Chapter 2

Software Development Life Cycle and Project Process

2.1 Selected SDLC Model

For CodeShield, an iterative and incremental approach is followed. The project has several components—frontend, backend, database, static analysis, ML pipeline, risk assessment and reporting—that can be developed, tested and integrated step by step.

2.2 SDLC Diagram

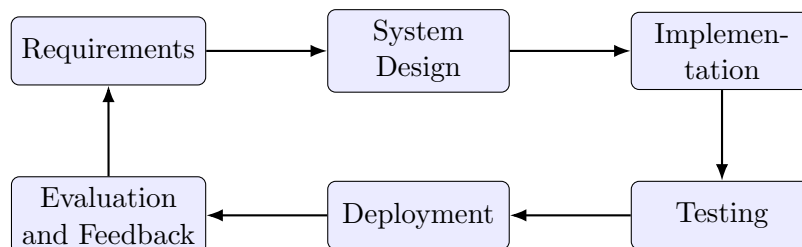


Figure 2.1: Iterative and incremental SDLC adopted for CodeShield.

2.3 Development Increments

1. **Increment 1:** Requirements, architecture and UI design.
2. **Increment 2:** React frontend and FastAPI backend.
3. **Increment 3:** Source-code parsing and static-analysis rules.
4. **Increment 4:** Dataset preparation and ML model development.
5. **Increment 5:** ML-backend integration and risk assessment.
6. **Increment 6:** Testing, evaluation and dashboard refinement.

Chapter 3

Requirements Analysis

3.1 Functional Requirements

ID	Requirement
FR1	The system shall allow a developer to submit source code.
FR2	The system shall preprocess and parse the submitted source code.
FR3	The system shall perform static-analysis checks.
FR4	The system shall apply the trained ML model to supported code.
FR5	The system shall identify supported vulnerability categories.
FR6	The system shall generate severity and confidence information.
FR7	The system shall report affected file and line information when available.
FR8	The system shall store scan information in the database.
FR9	The system shall display scan results through the dashboard.

3.2 Non-Functional Requirements

- **Performance:** Scans should complete within a practical time for the supported input size.
- **Reliability:** Invalid code and failed scans should be handled gracefully.
- **Security:** Submitted code should be processed with controlled access.
- **Maintainability:** Static-analysis rules and ML models should be modular.
- **Usability:** Results should be understandable to developers.

Chapter 4

System Architecture and Design

4.1 Use Case Diagram

The primary actor is the Developer, who submits code, starts a scan, and views results. The system internally performs preprocessing, static analysis, ML analysis, risk assessment and report generation.

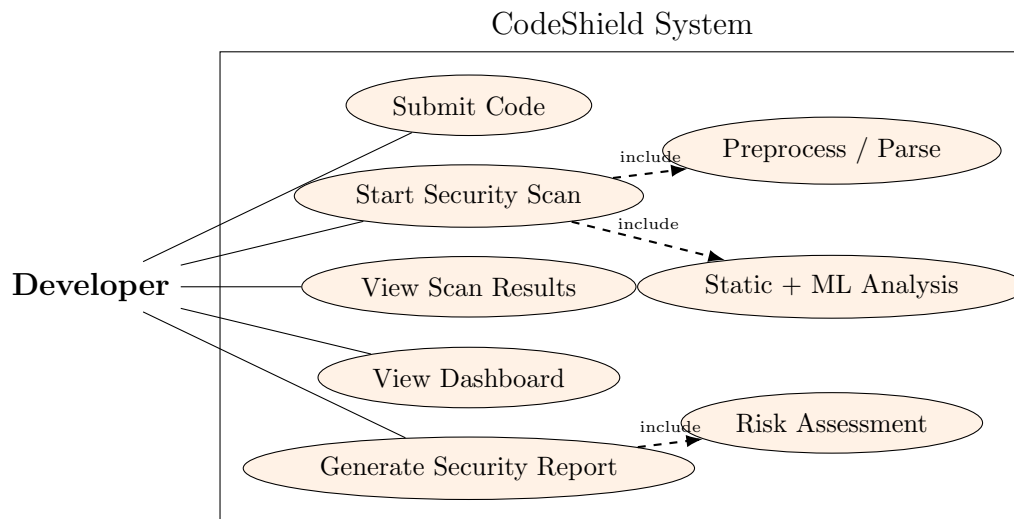


Figure 4.1: Use case diagram for CodeShield.

4.2 High-Level System Architecture

The React frontend sends the code to the FastAPI backend, which connects to the analysis pipeline. The pipeline performs code parsing, static analysis and ML-based analysis, followed by result fusion, risk assessment and reporting. Results are stored in PostgreSQL.

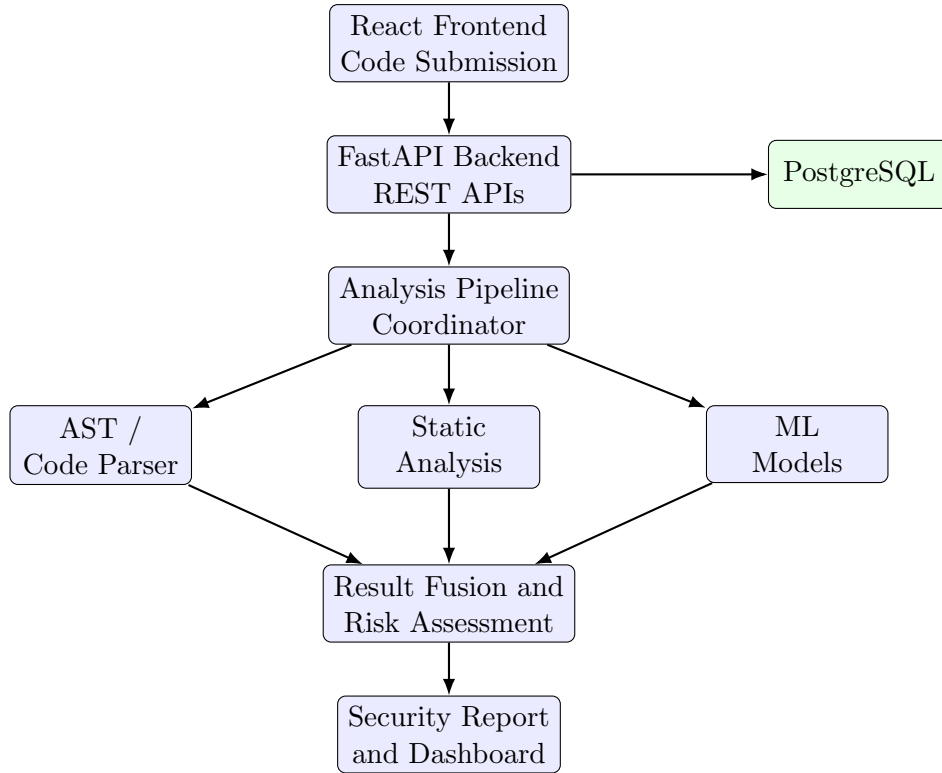


Figure 4.2: End-to-end CodeShield system architecture.

4.3 Core Scanning Workflow

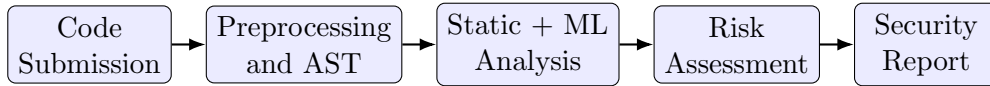


Figure 4.3: CodeShield core vulnerability-scanning workflow.

4.4 UML Class Diagram

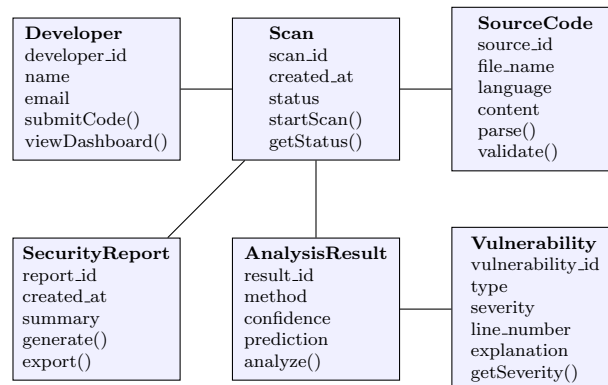


Figure 4.4: UML class diagram for the logical CodeShield software model.

4.5 Activity Diagram

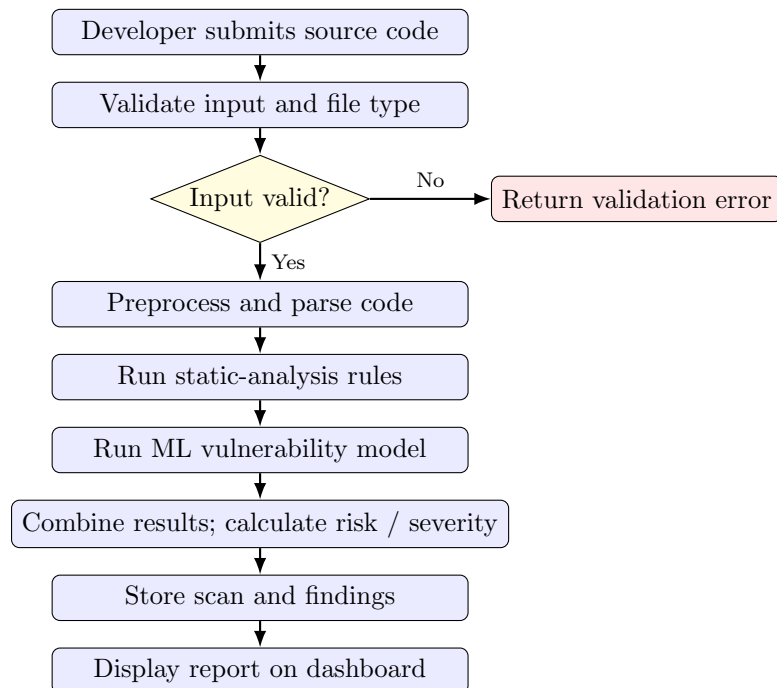


Figure 4.5: Activity diagram for a CodeShield security scan.

Chapter 5

Methodology and Design

5.1 Code Preprocessing

The submitted source code is first parsed to extract useful structures depending on the supported language, such as function calls, user-controlled input, database operations, file-system operations and command execution.

5.2 Static Analysis

Static analysis provides a deterministic layer based on predefined security rules. It identifies suspicious patterns without executing the program. For example:

```
query = "SELECT * FROM users WHERE id=" + user_id
```

A rule can identify direct concatenation of user-controlled data into an SQL query and generate a potential SQL-injection finding.

5.3 Machine Learning Pipeline

Along with static analysis, the ML pipeline predicts the vulnerability type of a Python code snippet:

Python Code $\rightarrow$ *TF-IDF* $\rightarrow$ *Linear SVM* $\rightarrow$ *Probability Calibration* $\rightarrow$ *Vulnerability Class + Confidence*

5.3.1 TF-IDF Feature Extraction

The code is treated as text and converted into numerical features using TF-IDF with word-level features, unigrams and bigrams (`ngram_range=(1,2)`) and L2 normalization.

5.3.2 Linear SVM Classifier

The classifier is a Linear Support Vector Machine (`LinearSVC`) with `class_weight=balanced` to handle class imbalance and regularization parameter $C = 2.0$. It is well suited to high-dimensional sparse TF-IDF features.

5.3.3 Probability Calibration and Confidence

`LinearSVC` produces decision scores rather than probabilities, so the classifier is wrapped with sigmoid probability calibration. During inference, the backend uses the calibrated `predict_proba()` output to select the highest-probability class and return its confidence.

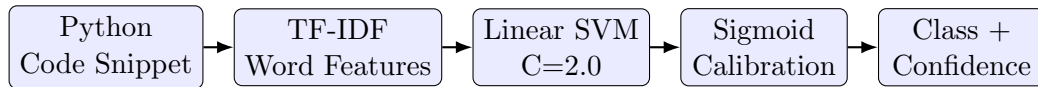


Figure 5.1: Current CodeShield multiclass ML inference pipeline.

5.4 Vulnerability Information and Risk Assessment

For every detected vulnerability, CodeShield records its type, severity, confidence, affected location and explanation. Findings are prioritized so that critical issues can be addressed first.

5.5 Result Fusion

Static-analysis findings and ML predictions are combined into a unified set of findings, which is then passed to risk assessment and reporting.

Chapter 6

Implementation

6.1 Frontend

The frontend is developed using React for code submission and for displaying vulnerability results through the dashboard.

6.2 Backend

The backend is implemented using Python and FastAPI. It manages scan requests, validates input, and connects the frontend with the analysis pipeline.

```
Frontend -> FastAPI -> Validation -> Analysis (Rules + ML)
        -> Risk Assessment -> Database -> Response
```

A health endpoint `GET /health` verifies that the service is running.

6.3 Database

PostgreSQL is used for persistent storage. The logical entities are Developer/User, Scan, Source Code, Vulnerability, Analysis Result and Security Report.

6.4 ML Model Integration

The trained model is integrated with the FastAPI backend so that predictions are generated during the scanning process. It applies the same TF-IDF transformation used in training, passes the features to the calibrated Linear SVM, and returns the predicted class with its calibrated confidence. This output is fused with the static-analysis findings and passed to risk assessment.

Chapter 7

Testing and Evaluation

7.1 Testing Methodology

The system was tested at both the software-system and ML levels.

- **Unit testing:** preprocessing, parsing, static rules, ML prediction, risk scoring and API endpoints.
- **Integration testing:** frontend → backend, backend → pipeline, pipeline → ML model, pipeline → PostgreSQL.
- **System testing:** complete scans on safe and known-vulnerable examples.
- **ML testing:** accuracy, precision, recall, F1-score and confusion matrix.

7.2 Performance Metrics

ML evaluation uses accuracy, precision, recall, F1-score and the confusion matrix. System-level evaluation includes scanning time and API response time.

7.3 Results Table

System-level values must be measured from the deployed backend and are therefore not fabricated here.

Table 7.1: CodeShield evaluation results currently available.

Metric	Achieved
Accuracy	90.00%
Macro F1-score	91.81%
Weighted F1-score	90.00%
Test samples	60
Average Scan Time	To be measured
API Response Time	To be measured

An accuracy of 90% corresponds to about 54 correctly classified snippets out of 60. Macro F1 gives equal importance to each vulnerability class, which is useful for an imbalanced dataset.

7.4 Model Configuration

Table 7.2: Configuration of the current CodeShield ML classifier.

Component	Configuration
Input	Python source-code snippet
Feature extraction	TF-IDF (word-level)
N-grams	Unigrams + bigrams
Normalization	L2
Classifier	LinearSVC, balanced class weights, $C = 2.0$
Calibration	Sigmoid probability calibration
Output	Vulnerability class + confidence

7.5 Confusion Matrix

The confusion matrix values are yet to be added from the final test run. They should be inserted from the same 60-sample run rather than estimated from aggregate metrics.

Actual	TP	FN
	FP	TN
Predicted		

Figure 7.1: Confusion matrix layout; replace the cells with measured values from the final test run.

Chapter 8

Results and Analysis

8.1 Example Vulnerability Detection

Consider the vulnerable query construction:

```
query = "SELECT * FROM users WHERE id=" + user_id
```

CodeShield reports the vulnerability type, severity, location, confidence and the reason the code is considered unsafe.

Table 8.1: Illustrative CodeShield vulnerability report.

Field	Example Result
Vulnerability	SQL Injection
Severity	High
Confidence	Model-generated calibrated probability
File	database.py
Line	12
Explanation	User-controlled input is concatenated into SQL

8.2 Challenges and Mitigation

Table 8.2: Major challenges and mitigation strategies.

Challenge	Mitigation
False positives	Combine ML predictions with static-analysis rules and confidence scores.
False negatives	Measure recall and clearly define supported vulnerability categories.
Limited labelled data	Use validated vulnerability datasets and controlled examples.

Chapter 9

Conclusion and Future Work

9.1 Summary of Work

CodeShield combines static analysis and machine learning to detect selected source-code vulnerabilities. The platform integrates a React frontend, a FastAPI backend, PostgreSQL storage, and an analysis pipeline containing parsing, static rules, ML classification, risk assessment and reporting.

9.2 Key Findings

The calibrated TF-IDF + Linear SVM model achieved 90% accuracy and a 91.81% macro F1-score on 60 test samples. Static analysis contributes deterministic findings, while the ML model adds data-driven classification with a confidence score.

9.3 Future Work

1. GitHub repository scanning.
2. Support for additional programming languages.
3. Improved ML models.
4. Better severity prioritization.
5. Automated remediation suggestions.
6. Scan history, asynchronous scanning and Docker-based isolated analysis.

9.4 Final Remarks

The project shows that conventional static analysis and machine learning can be combined rather than treated as competing approaches, helping developers identify and prioritize security weaknesses earlier in the development lifecycle.

Bibliography

- [1] National Institute of Standards and Technology, *Software Assurance Metrics and Tool Evaluation*, NIST.
- [2] OWASP Foundation, *OWASP Benchmark Project*, OWASP.
- [3] K. Chhabra et al., *Big-Vul: A Large-Scale Vulnerability Dataset*, GitHub repository and associated research resources.
- [4] National Institute of Standards and Technology, *Juliet Test Suite*, Software Assurance Reference Dataset.
- [5] FastAPI Documentation, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*.
- [6] React Documentation, *React: The library for web and native user interfaces*.